Heaps

Insertion:

[10, 20, 30, 40, 50, 60, 70]

Heapify-up:

[10

[10,20

[20,10,30

[30, 10, 20, 40

[30, 40, 20, 10

[40, 30, 20, 10, 50

[40, 50, 20, 10, 30

[50, 40, 20, 10, 30, 60

[50, 40, 60, 10, 30, 20

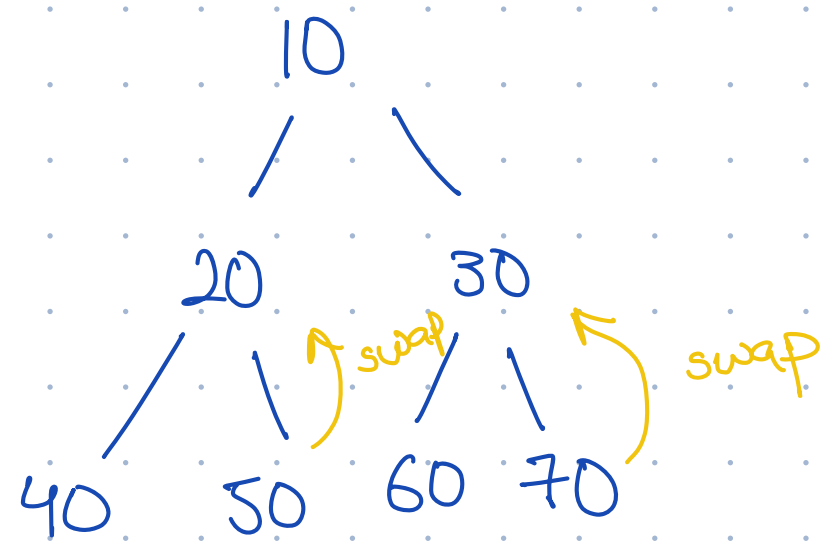
[60, 40, 50, 10, 30, 20, 70

[60, 40, 70, 10, 30, 20, 50

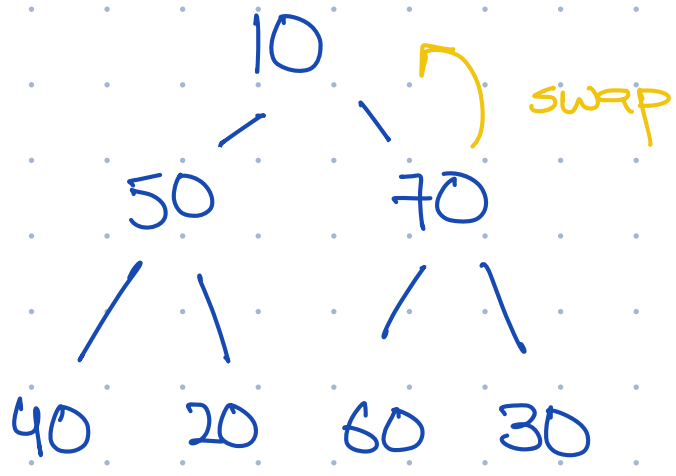
[70, 40, 60, 10, 30, 20, 50]

Heapify – down

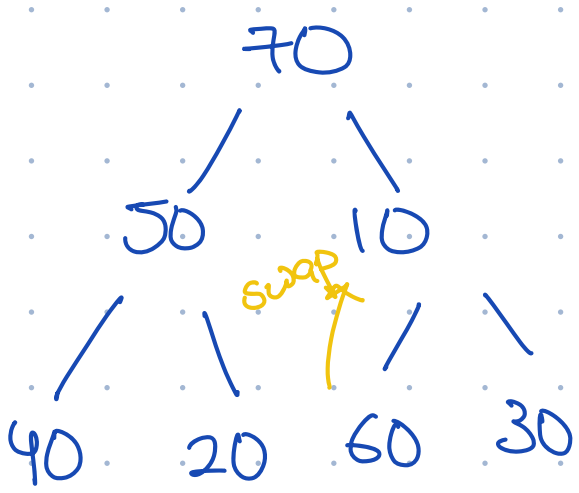
1	2	3	4	5	6	7
10	20	30	40	50	60	70



[10, 50, 70, 40, 20, 60, 30]



[70, 50, 10, 40, 20, 60, 30]



[70, 50, 60, 40, 20, 10, 30]

NOTE: both methods gave different heaps but they

are both valid.

Deletion:

1 2 3 4 5 6 7

[70, 50, 60, 40, 20, 10, 30]

[30, 50, 60, 40, 20, 10 | 70]

[60, 50, 30, 40, 20, 10 | 70]

[10, 50, 30, 40, 20 | 70, 60]

[50, 10, 30, 40, 20 | 70, 60]

[50, 40, 30, 10, 20 | 70, 60]

[20, 40, 30, 10 | 70, 60, 50]

[40, 20, 30, 10 | 70, 60, 50]

[10, 20, 30 | 70, 60, 50, 40]

[30, 20, 10 | 70, 60, 50, 40]

[10, 20 | 70, 60, 50, 40, 30]

[20, 10 | 70, 60, 50, 40, 30]

[10 | 70, 60, 50, 40, 30, 20]

[70, 60, 50, 40, 30, 20, 10]

Greedy Algorithms

Scenario: A delivery company transports standard packages and oversized packages. A standard package yields a profit of $5, weighs 2 kg, and takes up 1 cubic meter of space. An oversized package yields a profit of $12, weighs 5 kg, and takes up 4 cubic meters of space. The delivery truck has a maximum weight capacity of 500 kg and a volume capacity of 300 cubic meters. The company wants to determine the quantity of each package type to load onto the truck to maximize total profit.

Identify the following components based on the standard optimization problem definition:

1. The decision variables (x)
2. The objective function ($f(x)$)
3. The feasible region (S)

item	Profit	c_i	v_i
1	5	2	1
2	12	5	4

$C = 500$
$V = 300 m^3$

2. $f(x)$ = max profit such that $\sum c_i \leq C$ & $\sum v_i \leq V$
1. item 1 & item 2
3. { item 2 x 91, item 1 x 250, & all other combinations don't violate $\sum c_i \leq C$ & $\sum v_i \leq V$ }

Scenario: A fitness center wants to schedule two types of classes: **Yoga** and **HIIT**.

- Each Yoga class generates **$50** in profit and requires **100** square feet of space.
- Each HIIT class generates **$80** in profit and requires **200** square feet of space.
- The center has a total of **1,000** square feet available.

P_i s_i S

- The gym manager can only staff a maximum of **8** classes total per day.

Your Task:

Identify the following components based on the definitions in your slides: +1

1. The decision variables (x).
2. The objective function ($f(x)$).
3. The feasible region (S).

similar to 0/1 knapsack.
greedy can't solve it.

1. Yoga & HIIT $\Rightarrow x_1, x_2$

2. max profit such that $\sum s_i \leq S$ & total classes ≤ 8

   $\max \; 50x_1 + 80x_2$

   greed factor: $\frac{P_i}{S_i}$

3. {HIIT x 8, Yoga x 8, & remaining combination}

   $50x_1 + 80x_2 \leq 1000$ given $x_1 + x_2 \leq 8$ $x_1 \geq 0$ & $x_2 \geq 0$

Available Coins: $[20, 15, 5, 1]$ (unlimited quantity)

Target Amount: 30

Your Tasks:

1. Execute the Greedy approach step-by-step to find the solution array, mimicking the dry-run format from your lecture slides.
2. After finding the greedy solution, state whether this specific outcome is the true optimal solution (fewest possible coins) for the target of 30.

1. [20, 5, 5]
2. [15, 15]

Scenario: You are a cashier with a limited till. Use the greedy approach to make change, but you must respect the inventory counts.

- **Available Coins:** * 10×1 (You have only one 10-cent coin)

- 7×2 (You have two 7-cent coins)
- 1×10 (You have ten 1-cent coins)

- **Target Amount:** 25

Still no guarantee that greedy will solve this.

Greedy: [10, 7, 7, 1]

Optimal: [10, 7, 7, 1]

Here is a set of activities. Your goal is to select the **maximum number** of non-overlapping events.

Activity	Start Time	End Time
A1	2	5
A2	1	3
A3	4	7
A4	6	8
A5	5	9

A2 1 3 ✓
A1 2 5 ✗
A3 4 7 ✓
A4 6 8 ✗
A5 5 9 ✗

Fractional Knapsack Practice

In this problem, you have a set of items, each with a weight (w_i) and a value (v_i). Your goal is to maximize the total value in your knapsack without exceeding its maximum capacity (W). Because this is the **Fractional** version of the problem, you are allowed to take a fraction of an item if it doesn't fit entirely. +3

Your Scenario

Knapsack Capacity (W): 25 kg

Item	Value (v_i)	Weight (w_i)	v/w
Item 1	40	10 kg	$40/10 = 4$
Item 2	42	7 kg	$42/7 = 6$
Item 3	25	5 kg	$25/5 = 5$

$$\begin{bmatrix} 2, & 3, & 1 \\ 18 & 13 & 3 \end{bmatrix} = 42 + 25 + 40 = 107$$

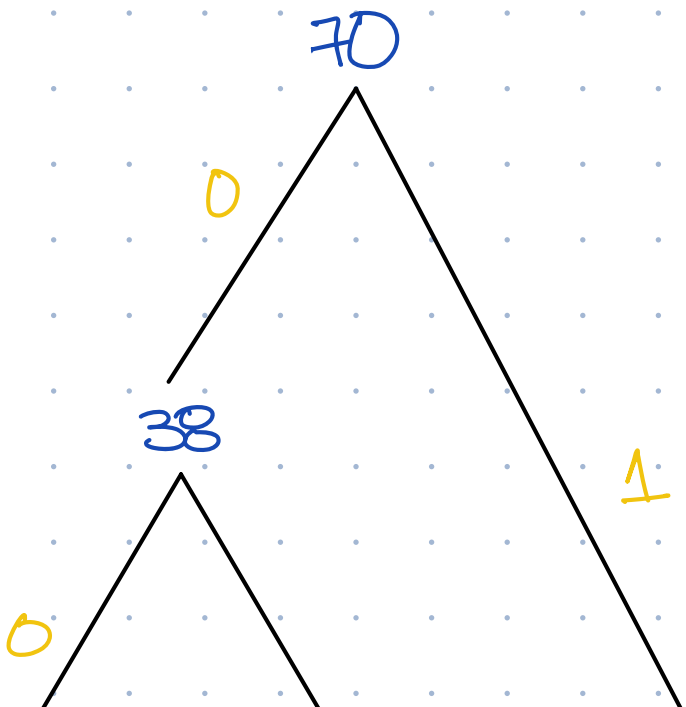
Practice Problem: Huffman Encoding

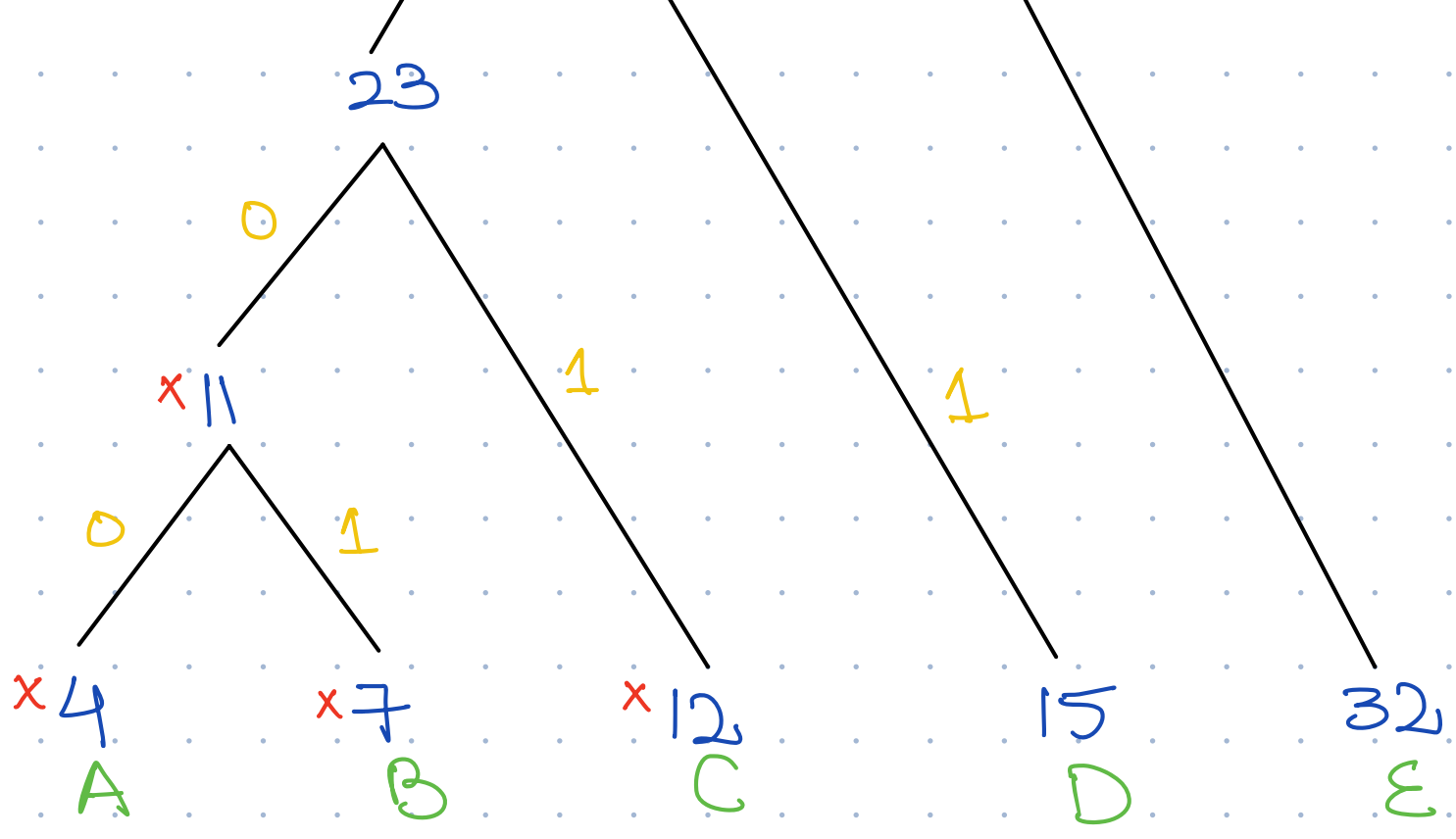
Character Set and Frequencies:

- **A**: 4
- **B**: 7
- **C**: 12
- **D**: 15
- **E**: 32

Message size without compression: $(4 + 7 + 12 + 15 + 32) \times 8$

560 bits





Character	code
A	0000
B	0001
C	001
D	01
E	1

Message size: $32 \times 1 + 2 \times 15 + 12 \times 3 + 7 \times 4 + 4 \times 4$

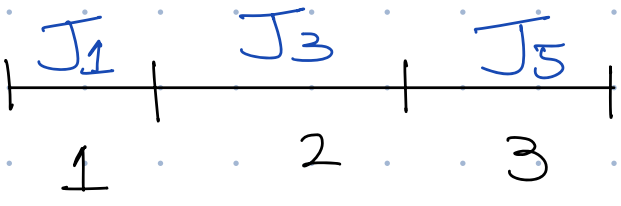
$+$

$5 \times 8 + (4 + 4 + 3 + 2 + 1)$

196 bits

Job	Profit	Deadline
J1	70	2
J2	50	1
J3	100	2
J4	20	3
J5	80	3

J_3	100	2
J_5	80	3
J_1	70	2
J_2	50	1
J_4	20	3



$= 100 + 80 + 70 = 250$

Dynamic Programming

0/1 Knapsack (Dynamic Programming) Practice

Parameters

- **Number of Items (n):** 4
- **Knapsack Capacity (W):** 8

Item (i)	Profit (p_i)	Weight (w_i)
1	4	3
2	5	4
3	8	5
4	3	2

Pi	wi		0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
4	3	1	0	0	0	4	4	4	4	4	4
5	4	2	0	0	0	4	5	5	9	9	9
8	5	3	0	0	0	4	5	8	8	8	12
3	2	4	0	0	3	4	5	8	8	11	12

$[\ 1\ (x_1)\quad 0\ (x_2)\quad 1\ (x_3)\quad 0\ (x_4)\ ]$

$3-3=0 \qquad 8-5=3$

Chain Matrix Multiplication

A_1 x A_2 x A_3 x A_4

3x2 2x4 4x2 2x5

d_0 d_1 d_2 d_3 d_4

m

i \ j	1	2	3	4
1	0	24	28	56
2		0	16	26
3			0	40
4				0

k

i \ j	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

$m[1,2] = m[1,1] + m[2,2] + (3 \times 2 \times 4)$

$1 \le k < 2 \quad = 0 + 0 + 24$

$= 24$

$m[2,3] = m[2,2] + m[3,3] + (2 \times 4 \times 2)$

$= 0 + 0 + 16$

$2 \le k < 3 \quad = 16$

$m[3,4] = m[3,3] + m[4,4] + (4 \times 2 \times 5)$

$3 \le k < 4 \quad = 0 + 0 + 40$

$m[1,3] = \min \begin{cases} k=1 \\ m[1,1] + m[2,3] + (3 \times 2 \times 2), \\ k=2 \\ m[1,2] + m[3,3] + (3 \times 4 \times 2) \end{cases}$

$1 \le k < 3$

$= \min [0 + 16 + 12,\ 24 + 0 + 24]$

$= 28$

$m[2,4] = \min \begin{cases} k=2 \\ m[2,2] + m[3,4] + (2 \times 4 \times 5), \\ k=3 \\ m[2,3] + m[4,4] + (2 \times 2 \times 5) \end{cases}$

$2 \le k < 4$

$= \min [0 + 40 + 40,\ 16 + 0 + 20]$

$= 36$

$$m[1,4] = \min_{1 \le k < 4} \begin{cases} k=1 & m[1,1] + m[2,4] + (3 \times 2 \times 5), \\ k=2 & m[1,2] + m[3,4] + (3 \times 4 \times 5), \\ k=3 & m[1,3] + m[4,4] + (3 \times 2 \times 5) \end{cases}$$

$$= \min[0 + 36 + 30, 24 + 40 + 60, 28 + 0 + 30]$$

$= 58$

$$[A_1 \times (A_2 \times A_3)] \times A_4$$

Iteration / substitution method

$$T(n) = T(n-1) + \log n \quad \text{——— M}$$

put $n = n-1$ in M

$$T(n-1) = T(n-2) + \log(n-1)$$

put $T(n-1)$ in M

$$T(n) = T(n-2) + \log(n-1) + \log n \quad \text{——— (i)}$$

put $n = n-2$ in M

$$T(n-2) = T(n-3) + \log(n-2)$$

put $T(n-2)$ in (i)

$$T(n) = T(n-3) + \log(n-2) + \log(n-1) + \log n$$

⋮ kth

$$T(n) = T(n-k) + \log(n-(k-1)) + \cdots + \log(n-1) + \log n$$

$n - k = 1$

$k = n-1$

$$T(n) = T(n-(n-1)) + \log(n-(n-1-1)) + \cdots + \log n$$

$$= T(1) + \log(2) + \cdots + \log n$$

$$= T(1) + \log(1) + \log(2) + \cdots + \log n$$

$$= T(1) + \log(1 \times 2 \times \cdots \times n)$$

$$T(1) + \log(n!)$$

$n! < n^n$

$\log(n^n)$

nlogn

frequency/Step count method

```
p = 0;
for(i = 1; p <= n; i++) {
    p = p + i;
}
```

i	p
1	1
2	1+2
3	1+2+3
...	
k	1+2+3+...k

$$\frac{k(k+1)}{2}$$

$k^2 \leq n$

$k \leq \sqrt{n}$

$O(\sqrt{n})$

```
p=0
for(i = 1; i < n; i = i * 2)
{
p++;
}
for(j = 1; j < p; j = j * 2)
{
    opertaion;
}
```

——— logn

```
}
```

$2^k \leq \log n$

$k \leq \log(\log n)$

```
void AlgorithmA(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i * i; j++) {
            for (int k = 0; k < n / 2; k++) {
                // Statement
            }
```

n — $n(n)$ — $n * n^2 * n$

i	j
0	0
1	1
2	4
3	9
4	16
⋮	
k	k^2

$k^2 < i^2$

$O(n^4)$

```
void AlgorithmB(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j * j < n; j++) {
            // Statement
        }
    }
}
```

n — $\sqrt{n}$

$n\sqrt{n}$

```
for (i = 1; i * i < n * n; i = i * 2) {
    for (j = 0; j < i * i; j++) {
        // Statement
    }
}
```

i: $2^0, 2^1, 2^2, \dots, 2^k$

j: $(2^0)^2, (2^1)^2, (2^2)^2, \dots, (2^k)^2$

$4^0 + 4^1 + 4^2 + \dots + 4^k$

$O(n^2)$

```
int k = 1;
for (int i = 0; i < n; i += k) {
    k++;
    printf("A");
}
```

i: $0, 2, 5, 9, \dots, m$

k: $1, 2, 3, 4, \dots$

$0 + 2 + 5 + 9 + \dots + m = \frac{m(m+1)}{2} - 1$

$m^2 < n$

$m < \sqrt{n}$

$O(\sqrt{n})$

```
for (int i = 1; i <= n; i *= 2) {
    for (int j = 1; j <= i; j++) {
        // Statement
    }
}
```

$2^0 \; 2^1 \; 2^2 \cdots 2^k$

$2^0 \; 2^1 \; 2^2 \cdots 2^k$

$= 2^0 + 2^1 + 2^2 + \cdots + 2^k$

$= 2^{k+1} - 1$

$O(2^k)$ $\quad k \le \log n$

$O(2^{\log n})$

$O(n)$

```
for (int i = 1; i * i <= n; i++) {
    for (int j = 1; j <= i * i; j++) {
        // Statement
    }
}
```

$k^2 < n \quad k < \sqrt{n}$

i
1
2
3
⋮
k

j
1^2
2^2
3^2
⋮
k^2

$$1^2 + 2^2 + 3^2 + \dots + k^2$$

$$\frac{k(k+1)(2k+1)}{6}$$

$$k^3$$

$$O(\sqrt{n})^3$$

```
int p = 0;
for (int i = 1; p <= n; i++) {
    p = p + i;
    for (int j = 1; j <= p; j *= 2) {
        // Statement
    }
}
```

i
1
2
3
⋮
k

p
1
1+2
1+2+3
$1+2+3+\dots k$

$$\frac{k(k+1)}{2}$$

$$k^2 \le n$$

$$k \le \sqrt{n}$$

$$\sqrt{n}\log(\sqrt{n})$$

j	
1	2^0
3	2^1
6	2^2
⋮	⋮
	2^m

$$2^m \le p$$

$$m \le \log_2(p)$$

```java
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j += i) {
        // Statement
    }
}
```

i
1
2
3
⋮
k

j
$1 \to n$
$2 \to n/2$
$3 \to n/3$
⋮
$k \to n/k$

$$n + \frac{n}{2} + \frac{n}{3} + \cdots \frac{n}{k}$$

$$n + \frac{n}{2} + \frac{n}{3} + \cdots + 1$$

$$n\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right)$$

$$n \sum \frac{1}{i}$$

$$n \log n$$

To convert a series sum into a Big-O complexity, you must recognize which of the four foundational mathematical patterns your iteration count is forming.

1. The Arithmetic Series (Linear Step)

When it appears: The inner loop runs an incrementing number of times (1, 2, 3, 4...).

- **Code Trigger:** `for (int j = 0; j < i; j++)` where `i++`
- **The Series:** $1 + 2 + 3 + 4 + \cdots + n$
- **The Formula:** $\frac{n(n+1)}{2} = \frac{n^2+n}{2}$
- **Big-O:** Drop the constants and the non-dominant term (n).
- **Result:** $O(n^2)$

2. The Geometric Series (Multiplicative Step)

When it appears: The inner loop bounds double, triple, or halve at each step (1, 2, 4, 8...).

- **Code Trigger:** `for (int j = 1; j <= i; j++)` where `i *= 2`
- **The Series:** $1 + 2 + 4 + 8 + \cdots + n$
- **The Rule:** A geometric series sum is always strictly dominated by its largest term. The sum of $1 + 2 + 4 + \cdots + 2^k$ is $2^{k+1} - 1$. If the last term is n, the total sum is $2n - 1$.
- **Big-O:** Drop the constant multiplier and the -1.
- **Result:** $O(n)$ → only if outer loop is bounded by n else 2^n

3. The Harmonic Series (Fractional Step)

When it appears: The inner loop runs n times, but skips by i each time, causing it to execute fewer times as i grows.

- **Code Trigger:** `for (int j = 1; j <= n; j += i)`
- **The Series for j:** $\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \cdots + \frac{n}{n}$
- **The Rule:** Factor out the n to get $n(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n})$. The sum of the fractions $1 + \frac{1}{2} + \frac{1}{3} \ldots$ is mathematically proven to be bounded by the natural logarithm $\ln(n)$.
- **Big-O:** Substitute $\log n$ for the fraction series.
- **Result:** $O(n \log n)$

4. Sum of Logarithms

When it appears: The inner loop does logarithmic work, and the outer loop runs linearly.

- **Code Trigger:** `for (int j = 1; j <= i; j *= 2)` where `i++`
- **The Series:** $\log(1) + \log(2) + \log(3) + \cdots + \log(n)$
- **The Rule:** According to logarithmic properties, $\log(a) + \log(b) = \log(a \cdot b)$. Therefore, the series equals $\log(1 \cdot 2 \cdot 3 \ldots n)$, which is $\log(n!)$. Using Stirling's approximation, the upper bound of $\log(n!)$ is $n \log n$.
- **Big-O:** Substitute the approximation.
- **Result:** $O(n \log n)$

```
for (int i = 1; i * i < n * n; i *= 2) {
    for (int j = 0; j < i * i; j++) {
        for (int k = 1; k * k <= j; k++) {
            // Statement
        }
    }
}
```

$O(n)$

$$n\left(1+\frac{1}{2}+\frac{1}{2^2}+\cdots+\frac{1}{2^k}\right)$$

$$n\sum_{i=0}^{k}\frac{1}{2^i}$$

$$\frac{n+\frac{n}{2}+\cdots+\frac{n}{2^k}}{}$$

$$\frac{n}{2^0}\quad\frac{n}{2^1}\quad\frac{n}{2^2}\quad\cdots\quad\frac{n}{2^k}$$

$$n\cdot\frac{1}{2^i}$$

$$i=0\to k$$

```
for (int i = n; i > 0; i = i / 2) {
    for (int j = 0; j < i; j++) {
        // Statement
    }
}
```

i	2^0	$(2^0)^2$	1
	2^1	$(2^1)^2$	2
	2^2	$(2^2)^2$	3
	⋮	⋮	⋮
k	2^k	$(2^k)^2$	$m^2 < n^2$

$4^0 + 4^1 + 4^2 + \cdots + 4^k = O(n^2)$ $n^2\sqrt{n}$

```
int p = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= p; j++) {
        // Statement
    }
    p = p * 2;
}
```

i	2^0	2^0
1	2^1	2^1
2	2^2	2^2
3	⋮	⋮
k	2^k	2^k

$2^0 + 2^1 + 2^2 + \cdots + 2^k$

$2^0 + 2^1 + 2^2 + \cdots + 2^n$

$2^{n+1} - 1$

$O(2^n)$

```
for(int i = 1; i <= n; i++) {
    for(int j = i; j <= n; j++) {
        for(int k = 1; k <= j; k *= 2) {
            // Statement
        }
    }
}
```

i	j	k
1	n	log(n)
2	n−1	log(n−1)
3	n−2	⋮
⋮	⋮	log(n−k)
k	(n−k)	

$$\frac{\log(n)+\log(n-1)+\cdots+\log(n-k)}{\log(n)+\log(n-1)+\cdots\log(1)}$$

$$\log(n!)$$

$$n\log n$$

$k n \log n$

$n^2 \log n$

```
for (int i = 2; i <= n; i *= i) {
    for (int j = 1; j <= i; j++) {
        // Statement
    }
}
```

i j

2
2^2
2^3
2^4
$\vdots$
2^k

2^1
2^2
2^3
2^4
$\vdots$
2^k

$2 + 2^2 + 2^3 + 2^4 + \cdots 2^k$

$O(n)$

$2 + 2^2 + \cdots + 2^{\log n}$

$2 + 2^2 + \cdots + n$

```
for (int i = 2; i <= n; i++) {
    for (int j = 2; j <= n; j = j * j) {
        for (int k = 1; k <= j; k++) {
            // Statement
        }
    }
}
```

1
2
3
4
$\vdots$
k

2^0
2^1
2^2
$\vdots$
2^k

k
2^0
2^1
2^2
$\vdots$
2^k

$2^0 + 2^1 + 2^2 + \cdots + 2^k$

$2^0 + 2^2 + \cdots + n$

$O(n)$

$O(n^2)$

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j * j <= i; j++) {
        // Statement
    }
}
```

n × √n

n√n

```
for (int i = 1; i <= n; i *= 2) {
    for (int j = 1; j <= n; j += i) {
        // Statement
    }
}
```

i: 1, 2¹, 2², ..., 2^k

j: n, n/2, n/2², ..., n/2^k

n + n/2 + n/2² + ... + n/2^k

n + n/2 + ... + 1

O(n)

```
int p = 0;
for (int i = 1; i <= n; i *= 2) {
    p++;
}
for (int j = 1; j <= p; j++) {
    for (int k = 1; k <= n; k *= 2) {
        // Statement
```

— log n

— log n

— log n

```
    }
}
```

$\log n + \log^2 n$

$O(\log^2 n)$

```
for (int i = 1; i <= n; i++) {
    int j = n;
    while (j > 0) {
        j = j - i; 1, 2, 3, 4
        // Statement
    }
}
```

n

$n + \frac{n}{2} + \frac{n}{3} + \cdots + \frac{n}{n}$

$n\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right)$

$n \log n$

```
int i = n;
while (i > 2) {
    i = sqrt(i);
    // Statement
}
```

i : $n^{1/2^0}$

$n^{1/2^1}$

$\vdots$

$n^{1/2^k}$

$n^{1/2^k} > 2$

$\frac{1}{2^k}\log(n) > \log_2(2)$

$\log n > 2^k$

$\log(\log n)$

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i * i; j++) {
        for (int k = 1; k <= j; k *= 2) {
            // Statement
        }
    }
}
```

n

i : $1, 2, 3, \dots, n$

n^2

j : $1^2, 2^2, 3^2, \dots, n^2$

k : $1 \to 2^0$, $2^2 \to 2^1$, $3^2 \to 2^2$, $\dots$, $n^2 \to 2^m$

$2^0 + 2^1 + \dots + 2^K$

$\sum_j \log j$

$n^3 \log n$

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j * j <= n; j++) {
        for (int k = 1; k <= n / j; k++) {
            // Statement
        }
    }
}
```

n

$n\sqrt{n}$

$n + \frac{n}{2} + \cdots + \frac{n}{2^k}$

$n + \frac{n}{2} + \cdots + \frac{n}{2^k}$

$n \log k$

$n^2 \log n$

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        for (int k = 1; k <= 10000; k++) {
            // Statement
        }
    }
}
```

i: 1, 2, 3, ⋮, k

j: 1, 2, 3, ⋮, k

$1 + 2 + 3 + \cdots + k$

$O(n^2)$

```java
for(int i = 1; i <= n; i++) {
    int j = 1;
    while(j <= n) {
        for(int k = 1; k <= j; k++) {
            // Statement
        }
        j = j + i;
    }
}
```

i	j	k
1	n	n
2	$n/2$	$n/2$
3	$n/3$	$n/3$
⋮	⋮	⋮
k	n/k	n/k

$$n + \frac{n}{2} + \frac{n}{3} + \cdots + \frac{n}{k}$$

$$n\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{k}\right)$$

$n \log n$

$n^2 \log n$